

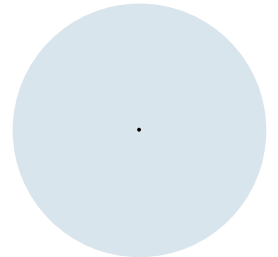
Einführung in die Programmierung (WIN+BIT)

Grundlagen Programmiersprachen

Prof. Dr. Johannes C. Schneider / Prof. Dr. Markus Eiglsperger



Programmieren



Programmieren

- Üblicherweise Abbildung eines realen Problems
 - „Wie komme ich am schnellsten von meinem Zuhause an die HTWG?“
 - „Wie ist der Gewinn meines Unternehmens am Ende des letzten Monats?“
 - ...
- auf für den Computer verständliche Anweisungen.
- Erfordert und schult
 - systematische Vorgehensweise
 - Abstraktionsvermögen
 - Kreativität
- auch für andere Bereiche (Nicht-Informatik).



Programmieren – Beispiel

Problem: Buchstabe e in einem Wort oder Satz zählen

"640K is more memory than anyone will ever need" (wird Bill Gates gesprochen)

→ Wie viele Vorkommen von e?

```
jshe11>
```

Lösung Mensch: Geht die Zeichen von links nach rechts durch und zählt, wenn immer er/sie ein e sieht.



programmieren
= übersetzen

Lösung Computer: Bekommt als Eingabe eine Zeichenkette (den Satz). In Programmiersprache schreiben:

- Variable zum Zählen der e's definieren
- Zeichenkette in einzelne Zeichen zerlegen
- durch die Zeichen durchgehen (Schleife)
- in einem bestimmten Fall (Verzweigung), nämlich, wenn das aktuelle Zeichen ein e ist, den Zähler erhöhen
- Am Schluss den Wert des Zählers ausgeben

IEEE Language Ranking 2021

Rank	Language	Type	Score
1	Python	  	100.0
2	Java	  	95.4
3	C	  	94.7
4	C++	  	92.4
5	JavaScript		88.1
6	C#	  	82.4
7	R		81.7
8	Go		77.7
9	HTML		75.4
10	Swift	 	70.4

(Bild: IEEE Spectrum)

PYPL PopularitY of Programming Language

Worldwide, Oct 2021 compared to a year ago:

Rank	Change	Language	Share	Trend
1		Python	29.66 %	-2.1 %
2		Java	17.18 %	+0.8 %
3		JavaScript	8.81 %	+0.4 %
4		C#	7.3 %	+1.1 %
5	↑	C/C++	6.48 %	+0.7 %
6	↓	PHP	5.92 %	+0.1 %
7		R	4.09 %	+0.2 %
8		Objective-C	2.24 %	-1.2 %
9	↑	TypeScript	1.91 %	+0.1 %
10	↑↑	Kotlin	1.9 %	+0.3 %
11	↓↓	Swift	1.86 %	-0.6 %
12	↓	Matlab	1.58 %	-0.2 %
13		Go	1.49 %	+0.1 %
14		VBA	1.33 %	+0.1 %
15		Ruby	1.09 %	-0.0 %

(Quelle: <https://pypl.github.io/PYPL.html>)

Warum Java?



- Einfache, klare, objektorientierte Konzepte
- Syntax an C angelehnt. Viele ähnliche Sprachen, wie z.B. C++, C#, JavaScript, PHP
- statisch typisiert: Viele Fehler werden schon vom Compiler gefunden
- Nutzung in vielen Anwendungen, große Auswahl an Programmbibliotheken, hohe praktische Relevanz
 - Java Enterprise Applications
 - Spring / Spring Boot / Hibernate
- Von viele Betriebssystemen und Geräte-Plattformen unterstützt
- Basis für Android-Entwicklung
- Java ist (historisch gesehen) eine **imperative** Programmiersprache, die mittlerweile aber auch Konzepte der **funktionalen** Programmierung umsetzt (nicht Teil dieser Vorlesung → SWEN 1)

Beispiel



- „Hello World!“ dient als minimales Beispiel für ein Programm!
- Es gibt einen Text („Hello World“) auf dem Bildschirm aus

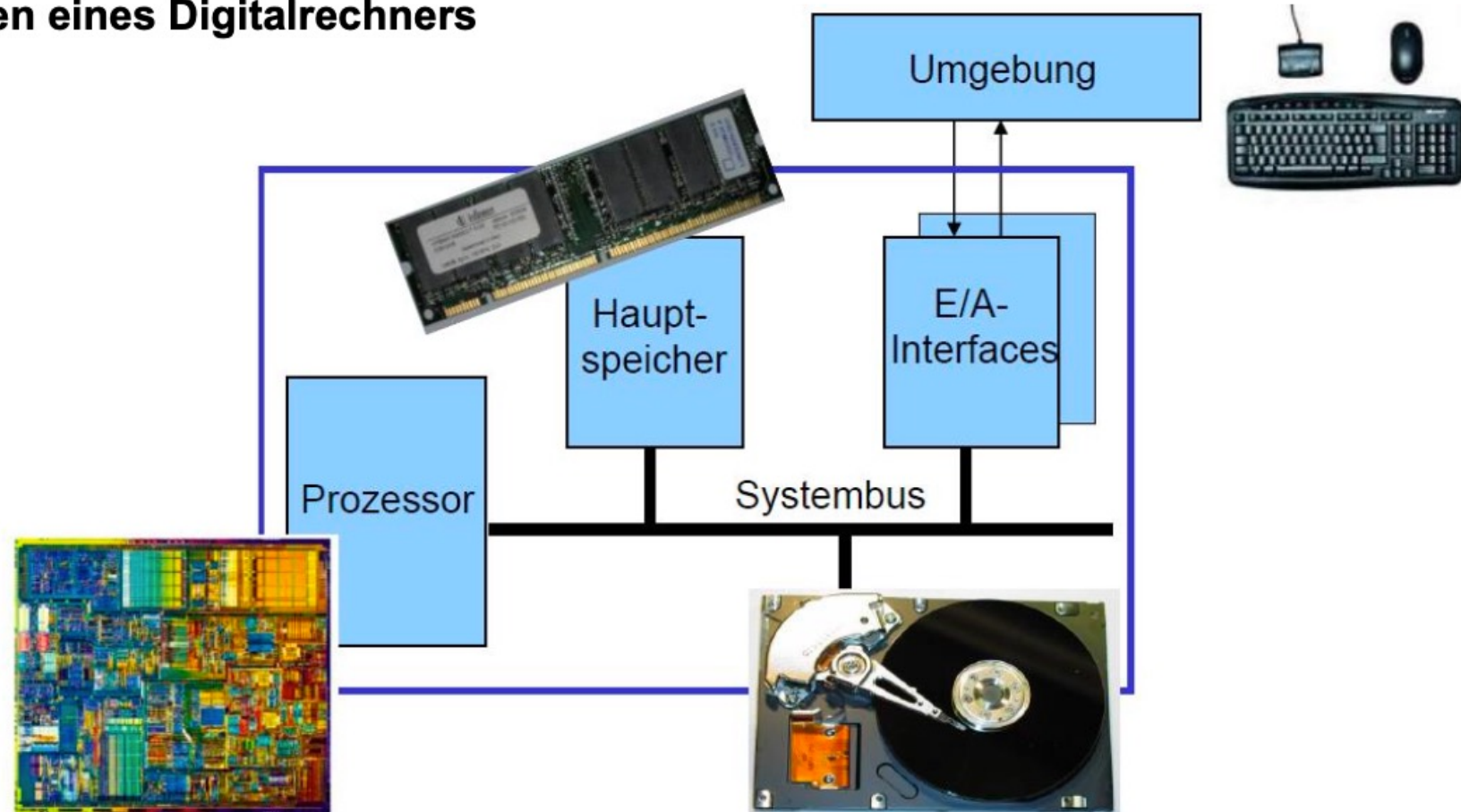
```
void main() {  
    System.out.println("Hello World");  
}
```



HelloStudents

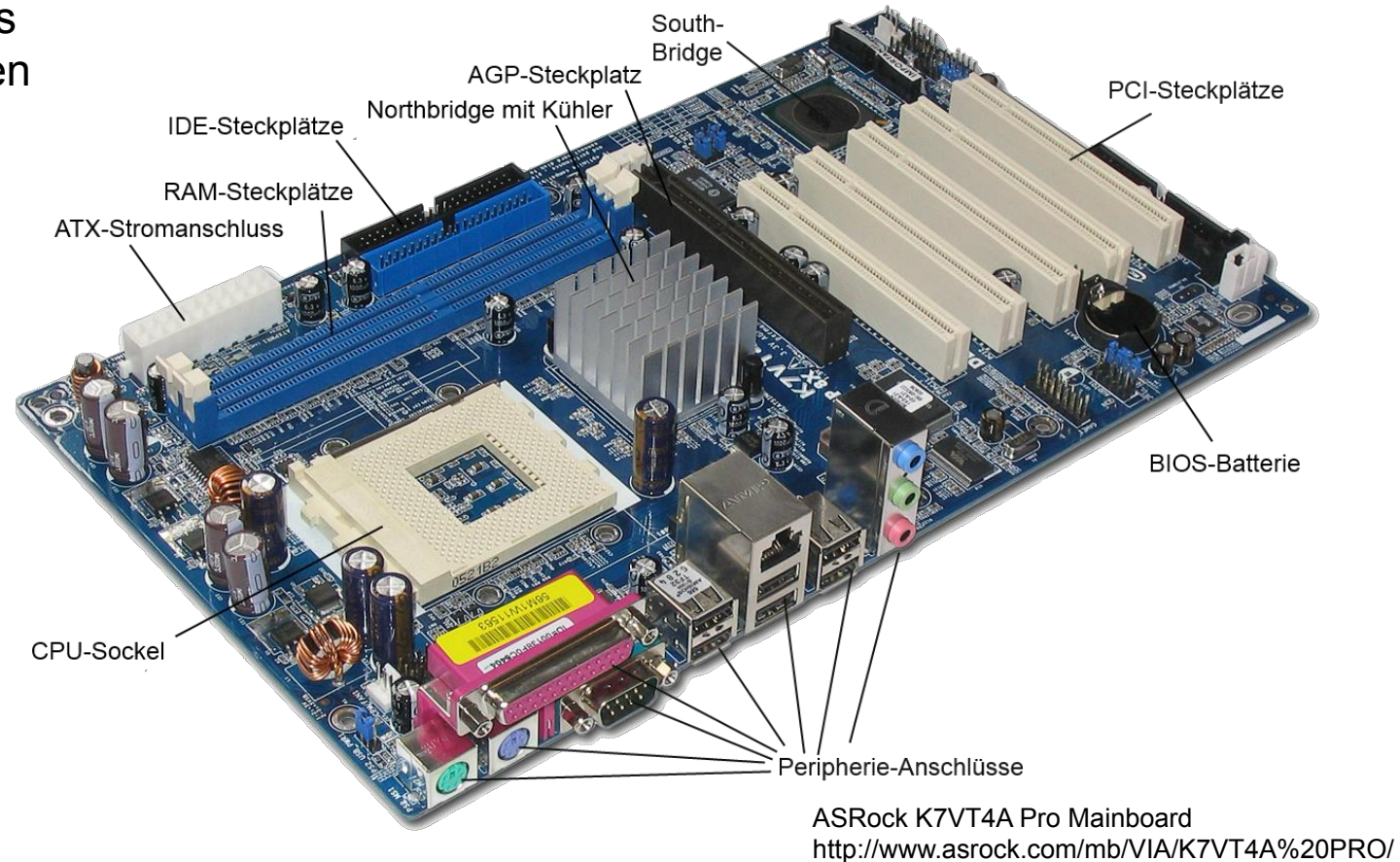
4. Grundlagen der Rechnerarchitektur

Komponenten eines Digitalrechners

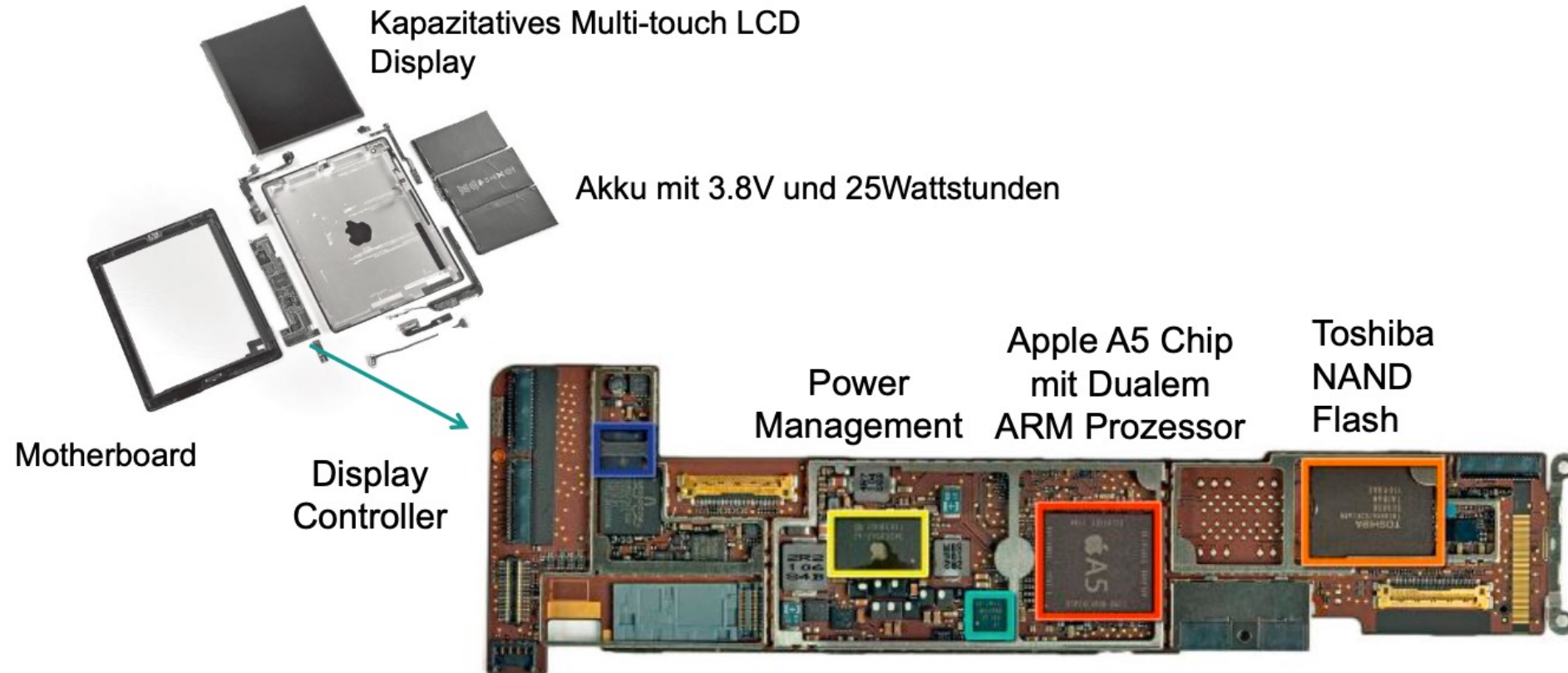


Die Vierte Generation (Rechner wie wir sie heute kennen)

- CPU mit Speicher, PCI-Bus und Peripherie-Anschlüssen
- North-Bridge und South-Bridge: Controller für Anbindung von Speicher und Peripherie-Geräten an CPU
- mehr im Laufe der Vorlesung



Das Innenleben eines Computers (Apple iPad 2)

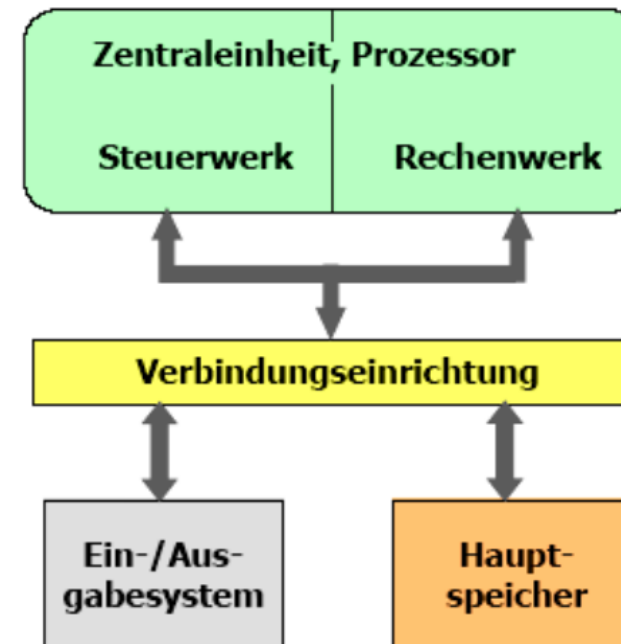


www.ifixit.com: iPad 2 Wi-Fi EMC 2415 Teardown

4. Grundlagen der Rechnerarchitektur

Komponenten eines von Neumann Rechners

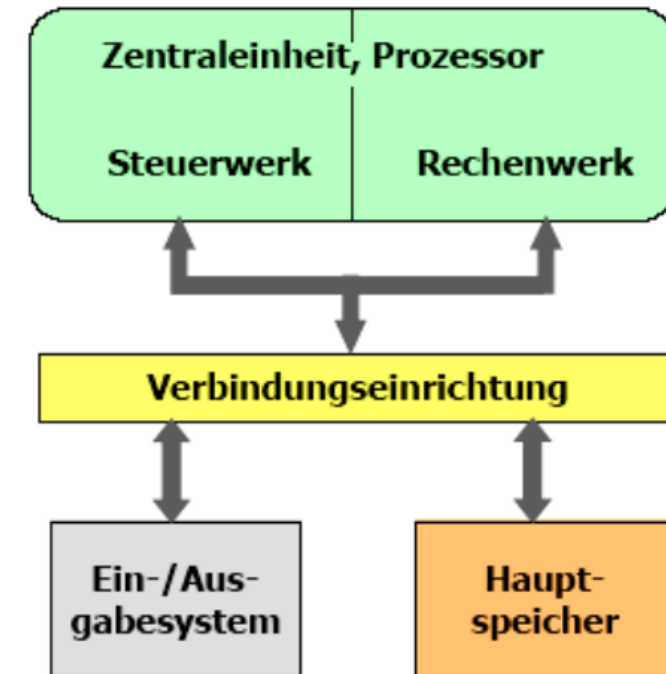
- **Zentraleinheit** (Central Processing Unit (CPU), Prozessor)
 - Verarbeitet Daten gemäß eines Programms
 - Besteht aus Leitwerk und Rechenwerk
- Leitwerk (Steuerwerk, Control Unit (CU))
 - Holt Befehle eines Programms aus dem Speicher
 - entschlüsselt sie und
 - Steuert ihre Ausführung in der verlangten Reihenfolge durch Steuer- und Synchronisierungs-Signalen
- Rechenwerk (Operationswerk, Ausführungseinheit, ALU)
 - Führt arithmetische/logische Operationen aus
 - Wird durch Steuersignale des Leitwerks beeinflusst
 - Liefert seinerseits Meldesignale an das Leitwerk zurück



4. Grundlagen der Rechnerarchitektur

Komponenten eines von Neumann Rechners

- **Verbindungsstruktur** (BUS)
 - BUS (Sammelschiene)
 - Systembus = Adressbus + Datenbus+ Steuerbus
- Adressleitung
 - Leitung auf der die Adressinformationen transportiert wird
 - unidirektional
- Datenleitung
 - Transportiert Daten und Befehle von/zum Prozessor
 - bidirektional
- Steuerleitung
 - Geben Steuerinformationen von/zum Prozessor (uni- oder bidirektional)

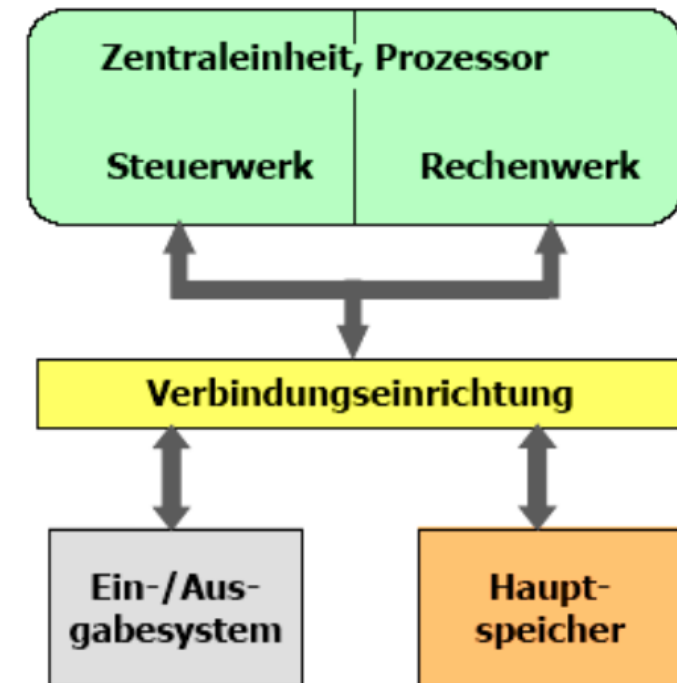


4. Grundlagen der Rechnerarchitektur

Komponenten eines von Neumann Rechners

▪ Ein-/Ausgabesystem (Peripheriegeräte)

- Geräte zur Eingabe von Daten und Programmen und zur Ausgabe der verarbeiteten Daten (Bildschirm, Drucker, Terminals, ...)
- Diese Geräte sind über Ein-/Ausgabe-Schnittstellen mit dem Rechner verbunden.
- Die Verbindung der Schnittstellen mit dem Prozessor (und zu den Peripheriegeräten) geschieht durch Adress-, Daten- und Steuerleitung.

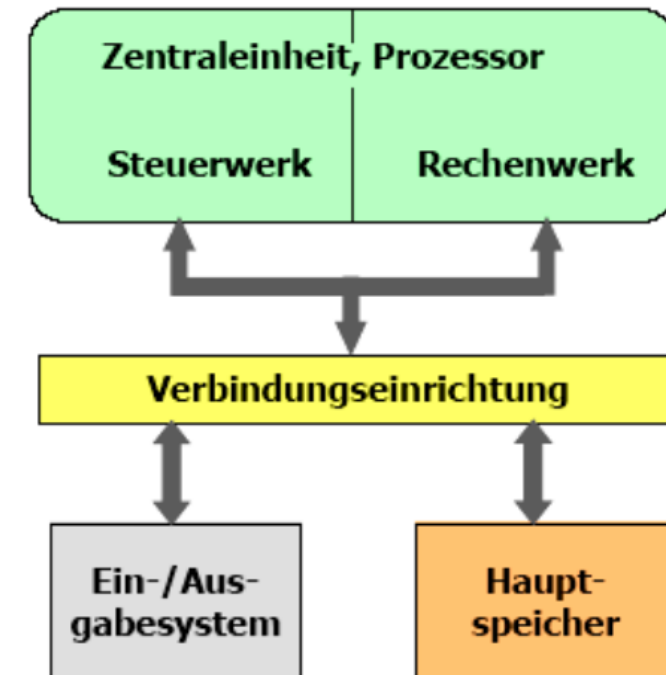


4. Grundlagen der Rechnerarchitektur

Komponenten eines von Neumann Rechners

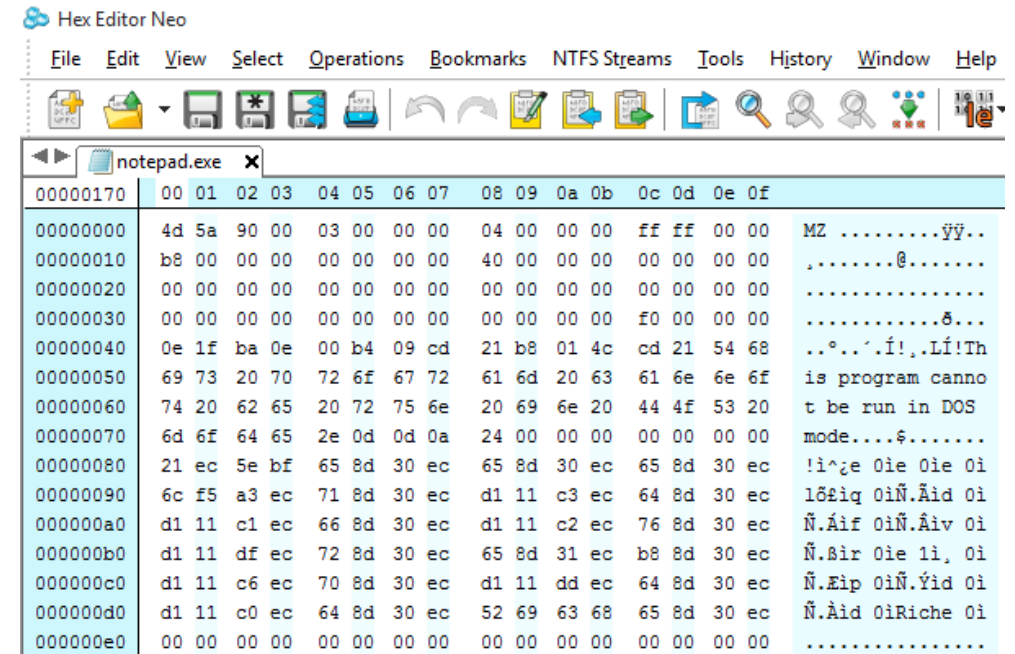
▪ Hauptspeicher

- Jede Speicherzelle ist eindeutig durch ihre Nummer (Adresse) identifizierbar.
- Dort werden Programme und Daten aufbewahrt (von-Neuman-Konzept)
- Den einzelnen Speicherzellen ist nicht anzusehen, welchen Typ von Information sie enthält
- Alternative: Havard-Architektur mit getrenntem Programm- und Datenspeicher
- Inhalt nach Abschaltung des Rechners flüchtig



Maschinencode

- Programme werden in einer **Programmiersprache** geschrieben
- Programmiersprache wird in **Maschinencode** übersetzt, also in Zahlenfolgen, die der Prozessor ausführen kann.



Unterschiede Maschinen-Code

- Es gibt viele **unterschiedliche Rechnerarchitekturen**, also auch viele **unterschiedliche Maschinencode-Befehlssätze**.
- Hier ein Beispiel vom [GNU C Compiler](#):

• AArch64 Options:		
• Adapteva Epiphany Options:		
• AMD GCN Options:		
• ARC Options:		
• ARM Options:		
• AVR Options:		
• Blackfin Options:		
• C6X Options:		
• CRIS Options:		
• CR16 Options:		
• C-SKY Options:		
• Darwin Options:		
• VxWorks Options:		
• x86 Options:		
• x86 Windows Options:		
• Xstormy16 Options:		
• Xtensa Options:		
• zSeries Options:		

3.19.59 x86 Options

These ‘-m’ options are defined for the

`-march=cpu-type`

Generate instructions for the machine specified by `cpu-type`, except where noted otherwise.

The choices for `cpu-type` are:

‘native’

This selects the CPU to generate code for the local machine under the current operating system.

‘x86-64’

A generic CPU with 64-bit registers.

‘x86-64-v2’

‘x86-64-v3’

‘x86-64-v4’

These choices for `cpu-type` are:

Since these `cpu-type` values are used to select the instruction set, they must be used in conjunction with the `-march` option.

‘i386’

Original Intel i386 CPU.

‘i486’

Intel i486 CPU. (No schedule branching instructions.)

‘i586’

‘pentium’

Intel Pentium CPU with register renaming.

‘lakemont’

Intel Lakemont MCU, based on the Pentium Pro core.

Assembler-Sprachen

- Die primitivsten Programmiersprachen sind die **Assembler**-Sprachen
 - Lesbare Namen für Maschinenbefehle, z.B.
 - `mov B, A`
(schiebt Wert von Adresse A nach Adresse B)
 - `xchg Op1, Op2`
(vertauscht Werte von Operand Op1 und Op2)
 - `jmp Label`
(springt zu dem mit Label markierten Befehl)
 - **abhängig von der Rechnerarchitektur** (Intel vs. ARM, 32bit vs. 64bit, ...)
→ Code **nicht portabel**
 - Sehr effizient, kann **sehr schnell** ausgeführt werden
 - **Mühsam** und fehleranfällig in Erstellung und Wartung

Assembler vs. Java

```
ASSUME  CS:CODE, DS:DATA
```

```
DATA    SEGMENT
Meldung db  "Hello World!"
        db  13, 10
        db  "$"
```

```
DATA    ENDS
```

```
CODE    SEGMENT
```

```
Anfang:
```

```
    mov ax, DATA
    mov ds, ax
    mov dx, OFFSET Meldung
```

```
    mov ah, 09h
    int 21h
    mov ax, 4C00h
    int 21h
```

```
CODE    ENDS
```

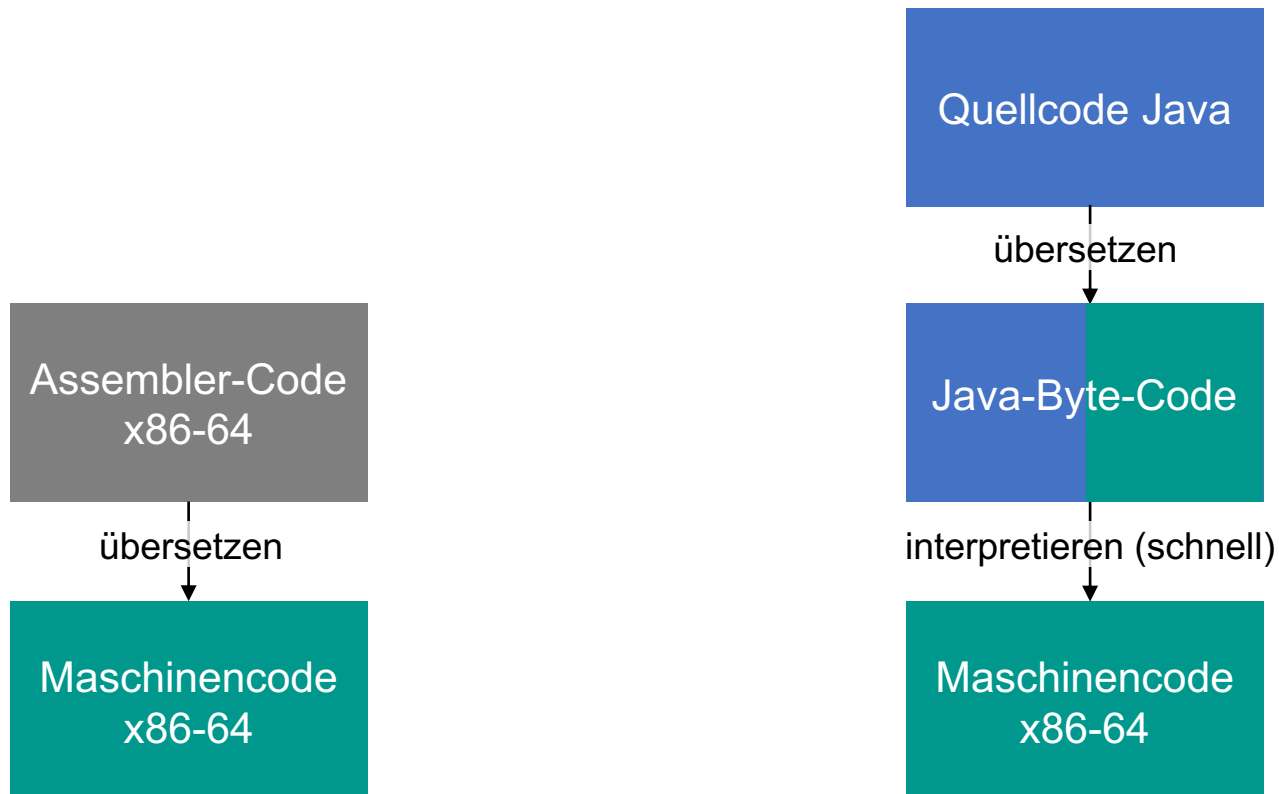
```
END     Anfang
```

```
System.out.println("Hello Students!")
```

Java Übersetzen und Ausführen

plattformunabhängig

effizient ausführbar



Programmiersprachen

- Heute fast nur noch **höhere** Programmiersprachen
 - **Rechnerunabhängig**
 - **Komfortablere** Programmierung durch **abstrakte** Konzepte, u.a.
 - Prozeduren, Subroutinen, Methoden
 - Module
 - Klassen, Schnittstellen, Vererbung
 - **Weniger effizient** als Assembler
 - Wesentlich schneller und **billiger in der Wartung** (Programme korrigieren, ändern, erweitern)
 - Es gibt **sehr viele** höhere Programmiersprachen
https://en.wikipedia.org/wiki/Category:High-level_programming_languages

Programme



- Programme erfüllen typischerweise einen **Zweck**, sie führen eine **Aufgabe** aus.
- Programme folgen oft dem dem EVA – Prinzip
 - **E** - Eingabe
 - **V** - Verarbeitung
 - **A** - Ausgabe

Einfache Java Programme mit main()

```
void main() {  
    // Eingabewerte  
    double fixkosten = 50000.0;  
    double verkaufspreis = 25.0;  
    double variableKosten = 15.0;  
  
    // Berechnungen  
    double deckungsbeitragProStueck = verkaufspreis - variableKosten;  
    double breakEvenMenge = fixkosten / deckungsbeitragProStueck;  
    double breakEvenUmsatz = breakEvenMenge * verkaufspreis;  
  
    // Ausgabe  
    System.out.println("Deckungsbeitrag pro Stück: " + deckungsbeitragProStueck);  
    System.out.println("Break-Even-Menge: " + breakEvenMenge);  
    System.out.println("Break-Even-Umsatz: " + breakEvenUmsatz);  
}
```

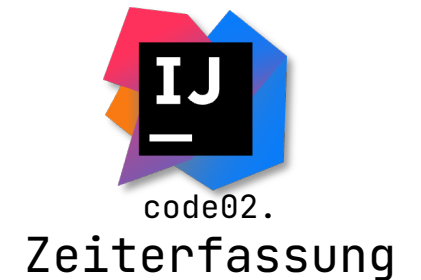


Weitere Beispiele:



- Aufgabe:

- **E** - Eingabe
- **V** - Verarbeitung
- **A** - Ausgabe



Interaktive Programme



- Das EVA (Eingabe – Verarbeitung – Ausgabe) Prinzip ist eine starke Vereinfachung, da dabei bereits zu Beginn alle Eingaben bekannt sein müssen.
- Dies ist z.B. bei Datenanalysen der Fall, aber in vielen anderen Fällen (z.B. Einkaufsprozess im Internet) nicht.
- Dort spricht man von **Interaktiven Programmen**, die nach jeder Benutzerinteraktion viele kleine EVA Routinen starten.
- Interaktive Programme können aber auch durch andere Ereignisse angestoßen werden, z.B. zeitgesteuert, oder über Sensordaten.



code02.
Ratespiel

Programmausführung

Ein Programm in einer höheren Programmiersprache kann

- übersetzt ("**kompiliert**") oder
- **interpretiert**

werden.

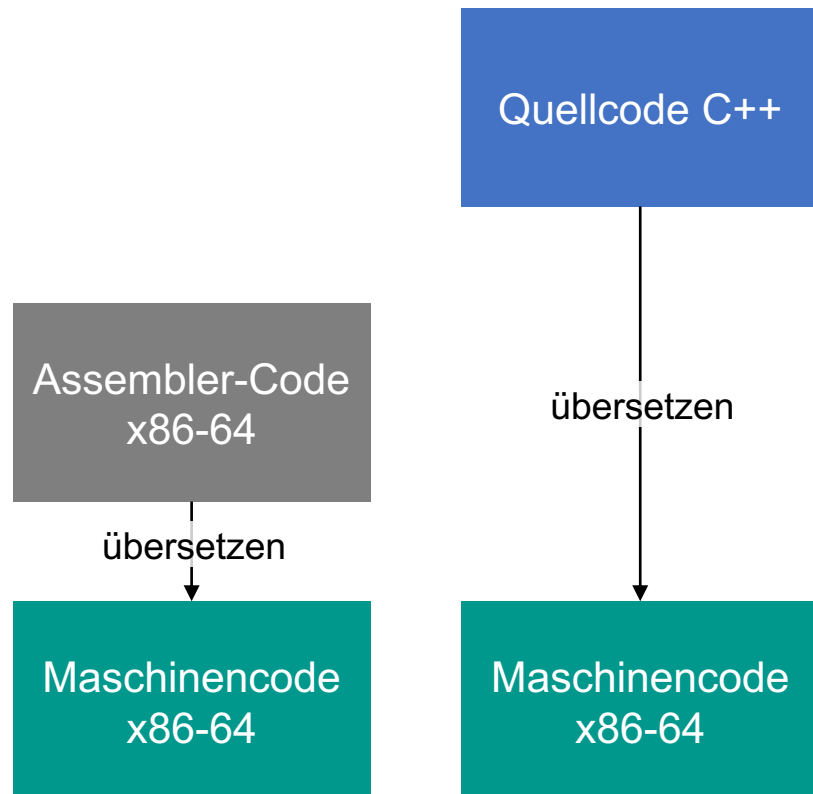
Kompilierung

- Compiler übersetzt Quellprogramm in **Maschinencode**
- Maschinencode wird gespeichert
- Maschinencode kann vom Rechner **direkt ausgeführt** werden
- **Effizient**
- Aber: Compiler und Maschinencode sind **rechnerabhängig**; portierter Code muss neu übersetzt werden ([verschiedene Architekturen C-Compiler](#))
- Typisch für C++

Vergleich

plattformunabhängig

effizient ausführbar



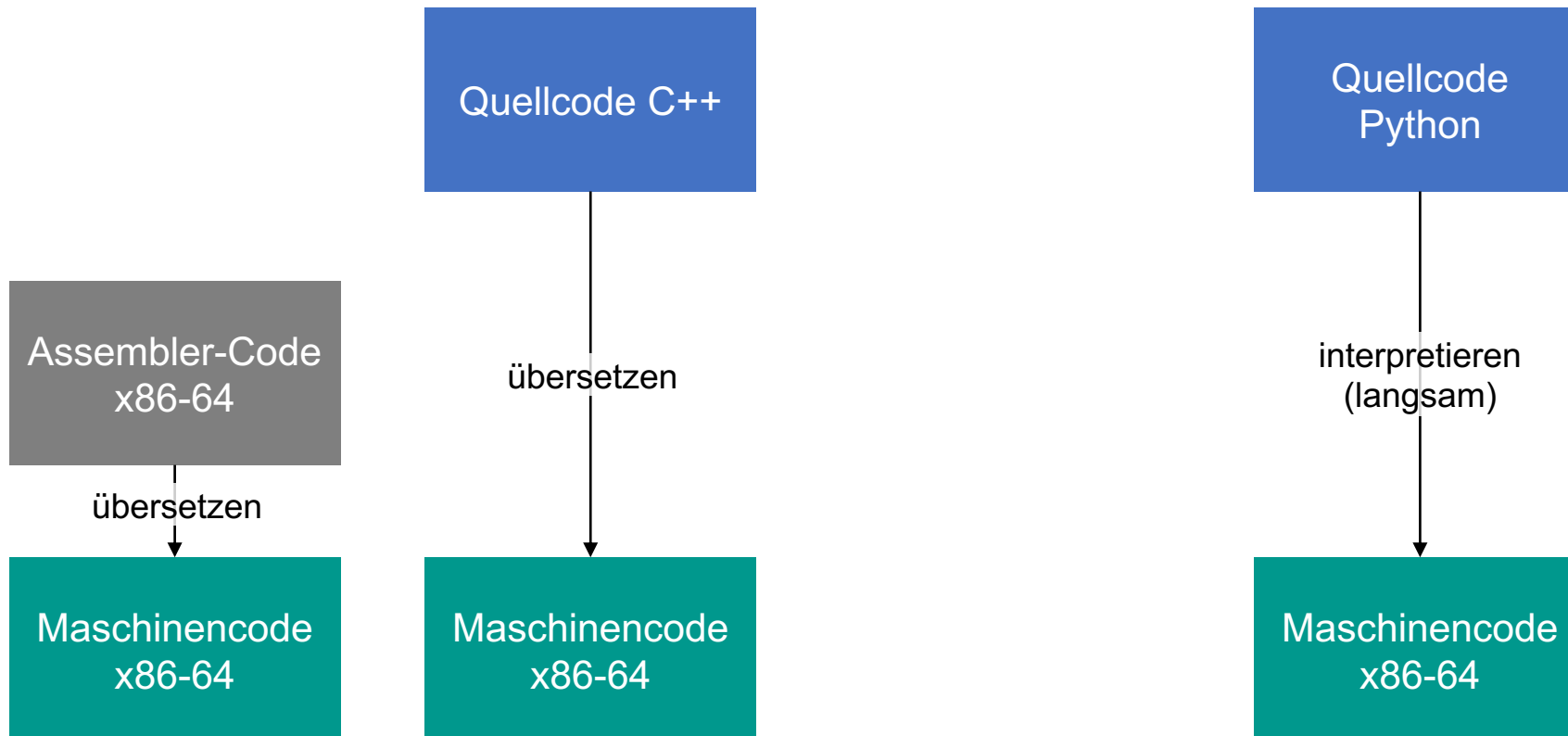
Interpretierung

- Quellcode wird von einem Interpreter Zeile für Zeile übersetzt und **direkt ausgeführt**
- Maschinencode wird nicht gespeichert
- Teile, die mehrfach durchlaufen werden, werden auch **mehrfach** übersetzt
- **Ineffizient**
- Vorteil: Programm kann direkt auf jede Zielmaschine **portiert** werden, die einen Interpreter für die Sprache besitzt
- Beispiel: Python, VBA-Makros

Vergleich

plattformunabhängig

effizient ausführbar



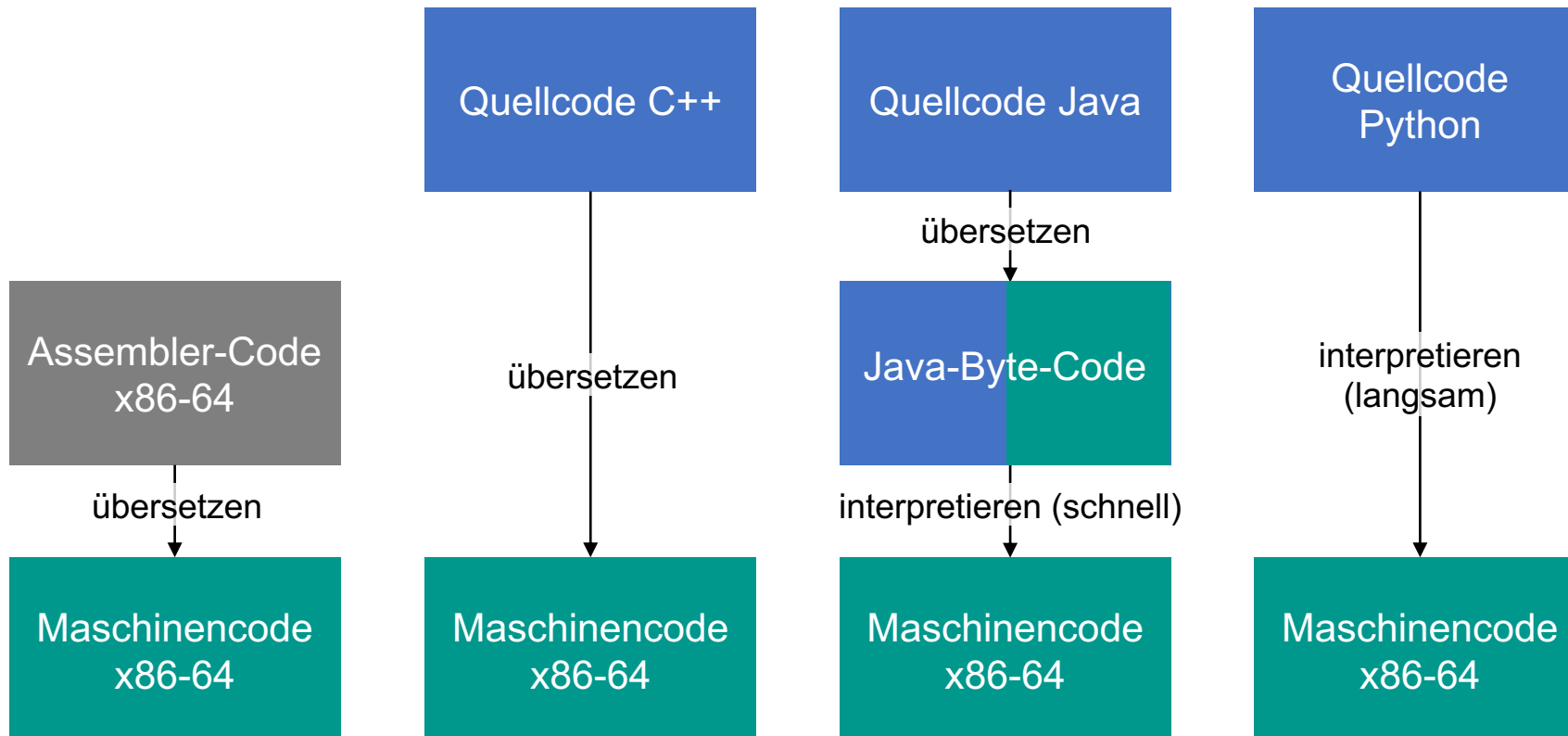
Mischform

- Compiler übersetzt Quellcode in **maschinennahes Zwischenformat** ("Bytecode")
- Bytecode wird auf Zielmaschine **interpretiert**
- Interpretieren **effizient** durch maschinennahe Struktur des Bytecodes
- Beispiele: Java, C#
- Bytecode-Interpreter = Java Virtual Machine (JVM)
- Kombination der Verfahren führt zu **portablem** Code, der **trotzdem schnell** ausgeführt werden kann

Vergleich

plattformunabhängig

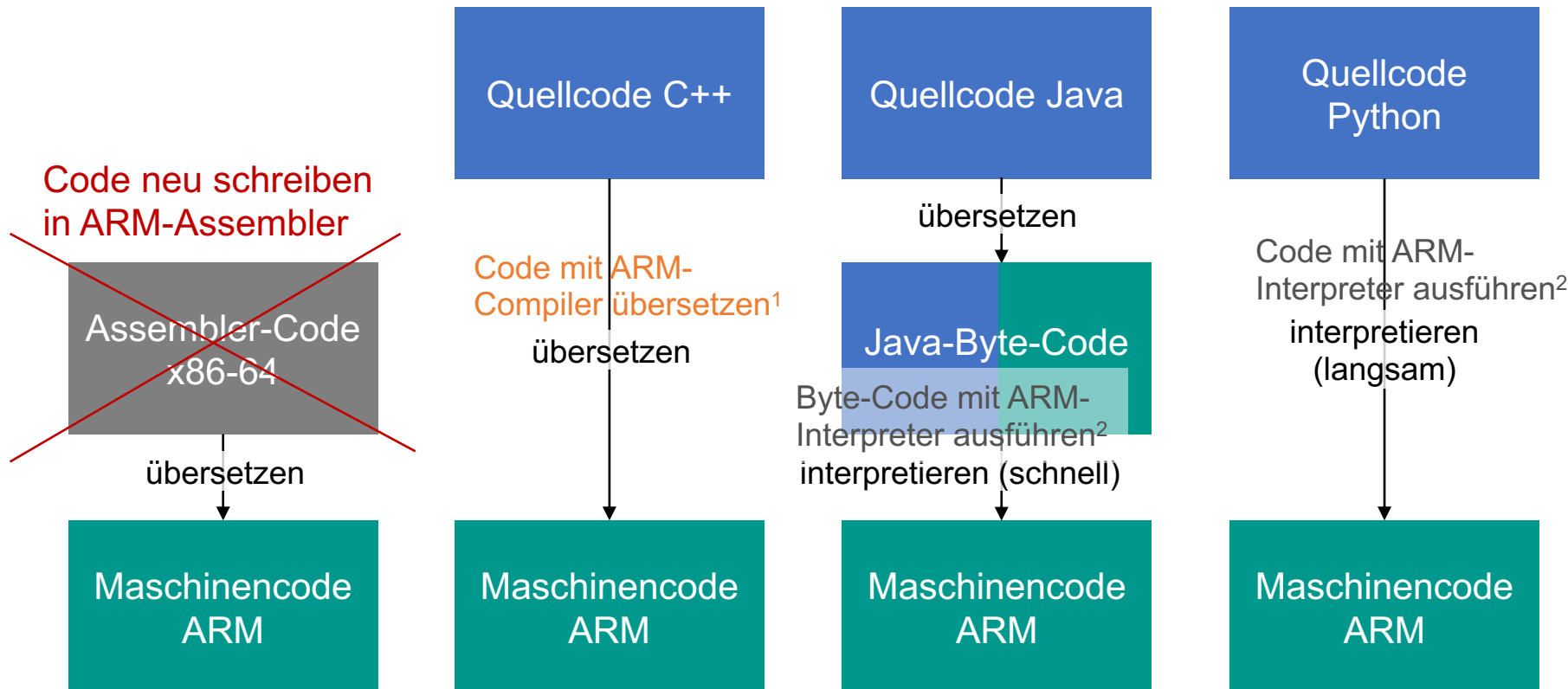
effizient ausführbar



Anpassungen für andere Ziel-Architektur (ARM)

plattformunabhängig

effizient ausführbar



¹Nur das eine, neu-übersetzte Programm funktioniert jetzt auf ARM

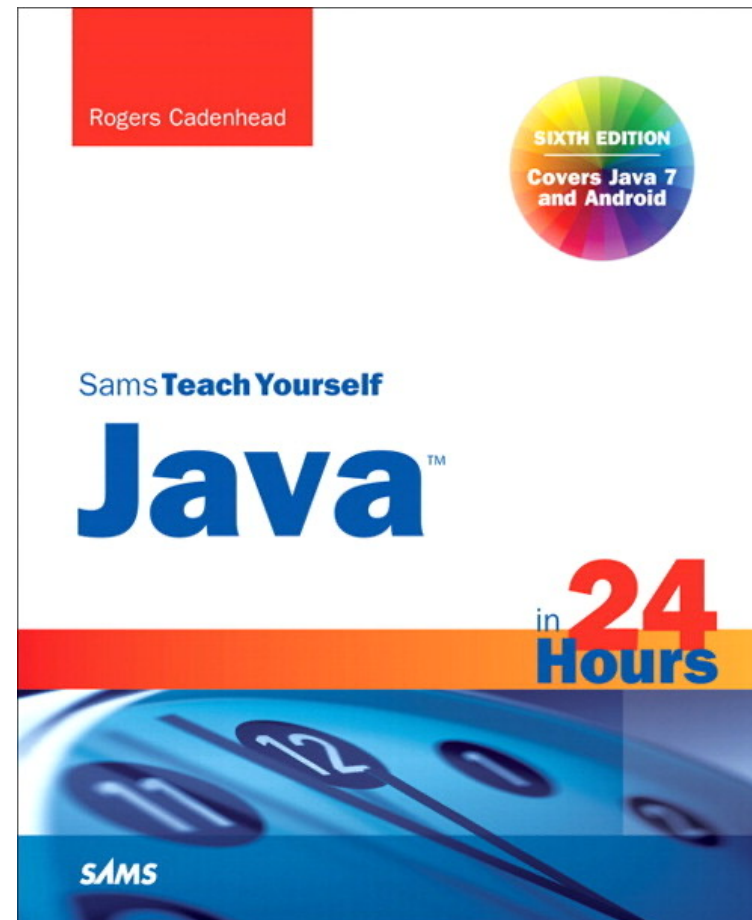
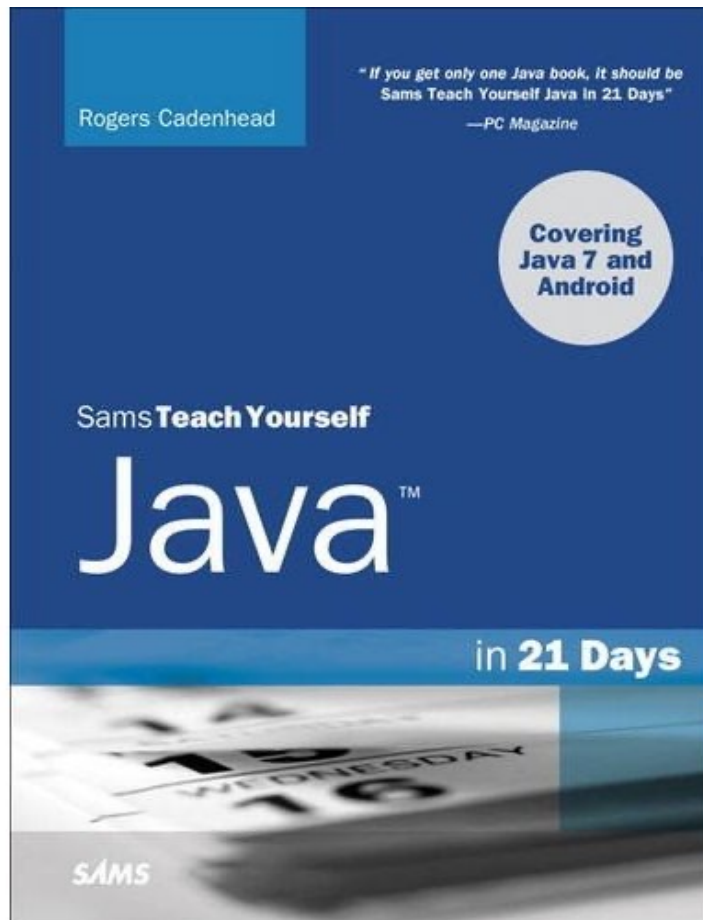
²Alle Programme, die es gibt, funktionieren jetzt auf ARM

Lernen einer neuen Sprache

- Üben, üben, üben!
- Kniffe (=Handwerk) lernt man nur durch praktische Erfahrung.
- Das dauert seine Zeit!
- Nicht aufgeben!
- **Lernen Sie verstehen, was Sie tun!**
Kein blindes Abtippen/Zusammenkopieren aus dem Internet!



Man braucht eigentlich viel länger!



Dranbleiben!

- Besser 10min am Tag üben als 1 Woche vor der Klausur
- Tools auf eigenem Rechner installieren → Jederzeit Übungsumgebung verfügbar
- Übungsaufgaben und –stunden nutzen!
 - Die Tutor:innen und ich helfen Ihnen gerne weiter und wiederholen Themen aus der Vorlesung mit Ihnen
- Problem am Anfang beim Lernen einer Programmiersprache: Fehler werden nicht verziehen (im Gegensatz zum Lernen einer natürlichen/Fremd-Sprache)
- Aber: Entwicklungsumgebung hilft einem beim Finden und Beheben der Fehler

Am Anfang deshalb viel experimentieren!

```
jshell>
```